DOTs in Action

Version Next, 2026-11-11

Table of Contents

Start with DOTs for Dummies . . . 1
Build on those concepts . . . 1
Your route . . . 1
Before you start . . . 2
Follow a repeatable rhythm . . . 3
Know where your work lives . . . 3
1. Run your first DOT . . . 4
  Install the released CLI . . . 4
  Check out the catalogue . . . 4
  Make a private learning workspace . . . 5
  Inspect before running . . . 6
  Get a real response . . . 6
  Stop and verify cleanup . . . 7
  Resume or retry without losing your work . . . 8
  Checkpoint . . . 10
2. Create an interface-aware DOT . . . 11
  Author the provided interfaces . . . 11
  Describe what Prometheus needs . . . 13
  Try the guided editor . . . 14
  Checkpoint . . . 15
3. Connect two DOTs . . . 15
  Create the use-case configuration . . . 15
  Create a fresh graph from the reviewed example . . . 16
  Understand the connection . . . 17
  Validate, lock and preview . . . 18
  Run and ask a meaningful question . . . 19
  Stop before extending . . . 20
  If the result differs . . . 21
  Checkpoint . . . 21
4. Grow the system with Grafana . . . 21
  Prepare the next real tool . . . 22
  Extend the existing graph . . . 23
  Set a password and start the extended system . . . 25
  Verify the whole chain . . . 25
  Clean up the complete graph . . . 28
  Checkpoint . . . 28
5. Take the pattern further . . . 29
  What you can now do . . . 29
  Grow deliberately . . . 29
  Operator capstone: authentication and policy . . . 30
  Contributor note: native cleanup audit . . . 31
  Turn a learning result into a contribution . . . 31

Learn to build a monitoring system you can explain and change. Run Prometheus and author a reusable definition in Lessons 1–2. Lessons 3–4 preview the metrics producer and Grafana composition; stop before live apply until the released cleanup-audit gate closes.

Your first visible result is small: **Prometheus answers its real API**. Next, your own DOT passes validation. The later previews explain how a two-tool query and Grafana would verify the same sample after the release gate closes. Each lesson separates its declaration, preview and live checks.

Start with DOTs for Dummies

Read DOTs for Dummies first. That introductory book supplies the concepts used here. This practical book assumes that background and takes you through installation, authoring, integration, live checks and cleanup. Keep both books available as you work; the concepts are explained in the introduction and applied in the lessons.

Build on those concepts

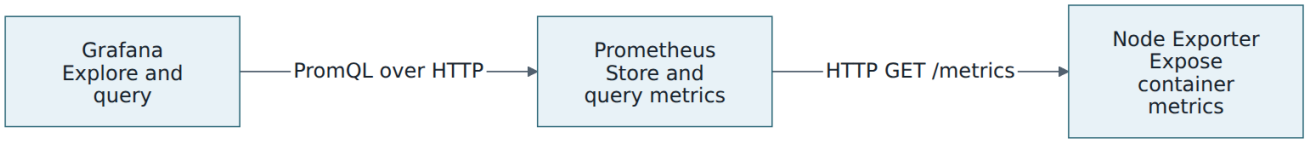


Figure 1. The system you will build: actual HTTP request direction

Grafana sends queries to Prometheus; Prometheus pulls metrics from Node Exporter. The CLI's topology uses a different, logical direction: a Wire points from the Anchor that **provides** an interface to the Anchor that **requires** it. Both views describe the same system. A Wire does not determine which tool sends a request.

The reusable DOT says what a tool can do and how it runs. Your composition says which instances work together for this particular monitoring use case. Scrape targets, datasource configuration and credentials belong to that use case.

Your route

Lesson	What you build	How you know it works
Start here: DOTs for Dummies	A mental model of the ecosystem and its main DOT-Domain building blocks	You can trace Contract → Anchor → service port and explain what a composition adds.
1. Run your first DOT	An existing Prometheus deployment	Its real API reports version 3.15.1; cleanup removes its owned resources.

Lesson	What you build	How you know it works
2. Create an interface-aware DOT	Your own Prometheus definition, with a pinned official Contract, named Anchors, configuration and two Docker deployment choices	The reusable profile validates it; inspection explains every declared part.
3. Connect two DOTs	Your Prometheus DOT plus the catalogue's Node Exporter	Preview only on the selected release; the later live checkpoint requires a labelled PromQL query returning `1`.
4. Grow the system	The same monitoring graph extended with Grafana	Preview only on the selected release; the later live checkpoint requires Grafana to return the same sample through its datasource.
5. Take the pattern further	A route into authenticated APIs, policy, other runtimes and Contract standards	You can identify which additional prerequisites and live checks each extension needs.

Choose your pace. Finish Lesson 1 for a first API response and Lesson 2 for a validated authored DOT. Read Lessons 3–4 as previews until the released cleanup-audit gate closes. Lesson 5 is an optional operator path with additional setup. Every live lesson ends with cleanup; stop there whenever you need a break. Downloads and startup depend on your machine and cache; take the time needed to review each result.

If you want to **watch before authoring**, use the guided demo on the documentation site instead. This tutorial is the hands-on route: you create and inspect the files rather than run an unattended installer.

Before you start

Use **Linux amd64**, Bash, Git, GitHub CLI, `curl`, `jq`, SHA-256 utilities and Docker with Compose. You need authorized access to the MARE repositories and release assets. The main four lessons need **no Java, Go, Task or Kubernetes**.

Use the released CLI `v0.4.0` with its manifest-pinned catalogue `2b17ee681fbe8eed6822834484f87149b32fc772` throughout. Lesson 1 includes installation. Do not switch to floating `main` halfway through: a version string alone does not identify later source changes. The catalogue may use the older single-root layout; identity-based lookups make these commands work without guessing paths.



Composition completion gate: the pinned CLI `0.4.0` does not provide the native resource-absence audit required by these lessons. Lessons 1 and 2 can be completed on this route. Read Lessons 3 and 4 as previews and stop before their live apply steps until this book pins a released CLI with the audit. Do not change binaries in an existing lab to obtain a different cleanup message.



Use an isolated development machine. The lab exposes disposable HTTP services; Docker publication can listen beyond localhost. Restrict access with your host firewall and use a lab-only Grafana password. This is not a hardened monitoring deployment. Node Exporter observes its container, not your physical host.

Validation establishes declared constraints and resolved interfaces. Only the API checks establish the particular live behavior exercised here. Neither result certifies every catalogue definition or every API of the selected tools.

Follow a repeatable rhythm

1. **Read the goal**, then run one complete Bash block at a time. Copy commands without adding a shell-prompt character. Keep one terminal and the matching CLI/catalogue throughout; variables carry your workspace into later lessons. A trailing `\` continues the same command on the next line. Copy the whole block, including those characters.
2. **Inspect your work.** Open the named file when a lesson asks for an edit. Keep YAML indentation exactly as shown. The longer shell blocks perform checked setup and verification; you do not need to memorize them.
3. **Check the result.** Validation checks declarations; a dry-run previews deployment. Continue to live execution only after those steps succeed.
4. **Clean up before moving on.** Use the lesson's original draft, lock, binary and cache. If a step fails, stop and use the resume and recovery guide.

Know where your work lives

Lesson 1 prints a private workspace path, referred to as `$LAB` throughout. The catalogue is a separate, pinned checkout; exercises do not edit it.

Inside your workspace	Purpose
`session.bash`	Your paths, ports and original CLI identity for resuming. It stores no password.
`base/dot.json`, then `prometheus/dot.json`	The generated scaffold and your enriched reusable tool definition from Lesson 2.
`composition.yaml`, `prometheus.yml`	Your two-tool graph and its scrape configuration from Lesson 3.
`dashboard.yaml`, `grafana/`, `grafana-*.yml`	Your extended graph, private Grafana definition and provisioning inputs from Lesson 4.
Adjacent `*.lock.json` files	Reviewed exact inputs created explicitly for each graph. Keep them for recovery.

The hands-on lessons end with a checkpoint and a short explanation exercise. Use those to test your understanding before moving to the next layer.

Begin with Lesson 1: run your first DOT.

1. Run your first DOT

Your first result is a running Prometheus service whose API reports the expected version. Start with reuse; you will author your own definition in Lesson 2.

Goal: deploy one existing DOT, verify its real API and remove only your instance. **You will learn:** tool identity, explicit deployment selection and the difference between a preview and a running service.

You need the prerequisites and isolated development host described in DOTs in Action. Run each Bash block only after the previous one succeeds. Keep the same terminal throughout the lessons. Parenthesized command blocks stop on errors inside their own shell; they do not close your terminal. A failed block is a stopping point, not permission to continue to deployment.

Install the released CLI

Authenticate with an account granted access to the private MARE repositories and releases. From a workspace directory:

```
set -o pipefail
gh auth login &&
gh auth setup-git &&
mkdir -p dot-quickstart-download &&
cd dot-quickstart-download &&
gh release download v0.4.0 --repo MARE-EU-Project/DOT-CLI \
  --pattern dotx-0.4.0-linux-amd64 --pattern SHA256SUMS &&
mkdir -p "$HOME/.local/bin" &&
sha256sum dotx-0.4.0-linux-amd64 |
  grep -Fx -f - SHA256SUMS &&
  install -m 0755 dotx-0.4.0-linux-amd64 \
    "$HOME/.local/bin/dotx" &&
export PATH="$HOME/.local/bin:$PATH" &&
dotx version &&
cd ..
```

The checksum must match the downloaded binary's exact filename and bytes; installation is skipped if its entry is missing or wrong. Stop on any error. Expected version: `dotx 0.4.0`. Add the `PATH` export to your shell profile for future sessions. The release also provides an SBOM and build/acceptance revision manifest.

Check out the catalogue

Only the independently cloneable catalogue is needed. Pin the acceptance revision recorded in this CLI release's manifest, then check your Docker access:

```
gh repo clone MARE-EU-Project/DOT-Repository &&
cd DOT-Repository &&
git checkout --detach 2b17ee681fbe8eed6822834484f87149b32fc772 &&
docker info &&
docker compose version
```

Make a private learning workspace

You are now in the **matching DOT-Repository checkout root**. The catalogue stays unchanged: exercises go in a new directory under your home directory. Save the printed path. If ports `18909`, `19041` or `13000` are occupied, stop and select unused ports before the corresponding live lesson. To use different ports throughout, set `DOTX_HTTP_PORT`, `TUTORIAL_PORT` and `DASHBOARD_PORT` to unused values before the workspace block; it saves your choices. Use those chosen ports in browser links instead of the displayed defaults.

```
# Start in the matching DOT-Repository checkout, in Bash.
set -o pipefail
DOT_REPOSITORY="$PWD" &&
PROMETHEUS_JSON="$(dotx repository resolve Prometheus@3.15.1 \
  --repository "$DOT_REPOSITORY")" &&
DOTX_TUTORIAL_CLI="$(command -v dotx)" &&
DOTX_TUTORIAL_CLI_SHA256="$(sha256sum "$DOTX_TUTORIAL_CLI" |
  cut -d ' ' -f 1)" &&
LAB_PARENT="${DOTX_TUTORIAL_WORKSPACE:-$HOME}" &&
LAB="$(mktemp -d "$LAB_PARENT/dot-zero-to-hero.XXXXXXXX")" &&
DOT_BUNDLE="$(dirname -- "$PROMETHEUS_JSON")" &&
DOT_NAME="hero-$RANDOM-$RANDOM" &&
DOTX_HTTP_PORT="${DOTX_HTTP_PORT:-18909}" &&
TUTORIAL_PORT="${TUTORIAL_PORT:-19041}" &&
DASHBOARD_PORT="${DASHBOARD_PORT:-13000}" &&
export DOTX_HTTP_PORT \
  XDG_CACHE_HOME="${XDG_CACHE_HOME:-$HOME/.cache}" &&
(umask 077; declare -p DOT_REPOSITORY LAB PROMETHEUS_JSON \
  DOT_BUNDLE DOT_NAME \
  DOTX_HTTP_PORT TUTORIAL_PORT DASHBOARD_PORT XDG_CACHE_HOME \
  DOTX_TUTORIAL_CLI DOTX_TUTORIAL_CLI_SHA256 > "$LAB/session.bash") &&
printf 'Your private workspace: %s\n' "$LAB" &&
printf 'Resume with: source %q\n' "$LAB/session.bash" &&
dotx version --verbose &&
git -C "$DOT_REPOSITORY" rev-parse HEAD
```

`repository resolve` finds an exact tool release. `LAB` holds your private drafts; it is not another catalogue root. `DOT_NAME` distinguishes this first deployment from someone else's instance. The saved `session.bash` contains paths and names, not credentials. After closing the terminal, source **your own** saved file to restore these variables before cleanup; do not create a new workspace instead.

Inspect before running

```
(
set -euo pipefail
printf '%s\n' \
  'Checking Prometheus and the selected Docker Compose deployment;' \
  'nothing starts yet...'
dotx validate "$DOT_BUNDLE" \
  --repository "$DOT_REPOSITORY" --profile reusable
dotx inspect "$DOT_BUNDLE"
dotx deploy "$DOT_BUNDLE" --deployment docker-compose \
  --name "$DOT_NAME" \
  --env "DOTX_HTTP_PORT=$DOTX_HTTP_PORT" --dry-run
printf '%s\n' \
  'Preview complete. Next: start this instance and check its API.'
)
```

Expected: validation succeeds, inspection lists deployment choices, and dry-run shows a Docker Compose command. Nothing has started yet.

The tool is Prometheus; `docker-compose` is the exact deployment ID. Choosing a different deployment does not create a different tool identity. Its definition already contains the pinned Compose document and image references; a random Compose file beside `dot.json` is not a deployment source.

Here is what Docker will do in this example. An **image** is the downloaded package containing Prometheus and what it needs to run. A **container** is a running instance of that image. The **Compose document** describes the services, network and storage to create together. This deployment stores its metrics in a **named volume**, separately from the container. Publishing a port connects Prometheus's service port `9090` to your chosen host port (`18909` by default), so a browser or `curl` on your machine can reach it. The cleanup step deletes this lab's metrics volume; the downloaded image stays cached.

Get a real response

If deployment or verification fails, use the cleanup block below before retrying. Keep the original binary, instance name and cache until cleanup is verified.

```
printf '%s\n' \
  'Starting Prometheus; Docker may download its pinned image...'
dotx deploy "$DOT_BUNDLE" --deployment docker-compose \
  --name "$DOT_NAME" \
  --env "DOTX_HTTP_PORT=$DOTX_HTTP_PORT"
```

Wait for readiness, then check the official build-information operation:

```
(
set -euo pipefail
ready=false
started=$SECONDS
next_notice=$SECONDS
printf 'Checking Prometheus readiness on port %s...\n' "$DOTX_HTTP_PORT"
for ((attempt=0; attempt<60; attempt++)); do
  if curl --fail --silent --max-time 2 \
    "http://127.0.0.1:$DOTX_HTTP_PORT/-/ready"; then
    ready=true
    break
  fi
  if ((SECONDS >= next_notice)); then
    printf 'Waiting for Prometheus: attempt %s/60, %ss elapsed\n' \
      "$((attempt + 1))" "$((SECONDS - started))"
    next_notice=$((SECONDS + 5))
  fi
  sleep 1
done
if ! $ready; then
  printf '%s\n' \
    'Prometheus readiness timed out.' \
    'Run the cleanup block before retrying.' >&2
  exit 1
fi
curl --fail --silent --show-error --max-time 5 \
  "http://127.0.0.1:$DOTX_HTTP_PORT/api/v1/status/buildinfo" |
  jq -e '.status == "success" and .data.version == "3.15.1"'
printf 'Verified: Prometheus 3.15.1 answered its official API.\n'
printf 'Open http://127.0.0.1:%s/\n' "$DOTX_HTTP_PORT"
)
```

Expected: `Prometheus Server is Ready.`, `true`, then a `Verified:` message confirming Prometheus `3.15.1` and printing its browser URL. The loop is bounded: up to 60 attempts, each with a two-second HTTP timeout and a one-second pause. Open the printed URL (Prometheus with the default port).

You have verified an actual service response, not just a valid definition. The selected Anchor names an operation; its Contract describes that operation; the service binding identifies the listening port.

Stop and verify cleanup



This Compose deployment's destroy operation removes its named metrics volume. The data is disposable. Do not use another person's instance name.

```
dotx destroy "$DOT_BUNDLE" --deployment docker-compose \
  --name "$DOT_NAME" &&
remaining="$(docker ps -aq \
  --filter "label=com.docker.compose.project=$DOT_NAME")" &&
test -z "$remaining" &&
remaining="$(docker network ls -q \
  --filter "label=com.docker.compose.project=$DOT_NAME")" &&
test -z "$remaining" &&
remaining="$(docker volume ls -q \
  --filter "label=com.docker.compose.project=$DOT_NAME")" &&
test -z "$remaining" &&
printf 'Cleanup verified: %s\n' \
  'Prometheus containers, network and volumes are absent.'
```

Expected: the instance stops and the three successful leftover queries print `Cleanup verified: Prometheus containers, network and volumes are absent.` Images remain cached. Keep `LAB` and the same terminal for the next lesson.

Resume or retry without losing your work

Paste the exact `source …/session.bash` command printed by the workspace block. Source only your own saved file. Then verify the original binary and restore its PATH before continuing:

```
# Source your own saved session.bash first, using the command printed in Lesson 1.
set -o pipefail
if [[ -n "${DOTX_TUTORIAL_CLI:-}" &&
      -n "${DOTX_TUTORIAL_CLI_SHA256:-}" &&
      -n "${LAB:-}" && -n "${DOT_REPOSITORY:-}" ]] &&
  test "$(sha256sum "$DOTX_TUTORIAL_CLI" | cut -d ' ' -f 1)" = \
    "$DOTX_TUTORIAL_CLI_SHA256"; then
  DOTX_TUTORIAL_BIN_DIR="$(dirname -- "$DOTX_TUTORIAL_CLI")" &&
  export PATH="$DOTX_TUTORIAL_BIN_DIR:$PATH" &&
  dotx version --verbose &&
  printf 'Resumed existing workspace: %s\n' "$LAB"
else
  printf '%s\n' \
    'Saved session is missing/incomplete, or its CLI changed.' \
    'Source your own session file and recover the recorded binary' \
    'before managing this workspace.' >&2
  false
fi
```

The saved session restores the repository, ports, instance name and cache location. If the binary check fails, stop and recover the original checksummed CLI before managing existing resources. Passwords are never saved: for a running Grafana instance, repeat Lesson 4's password prompt with the original lab password. Choose a new value only after that instance is cleaned up.

What already exists	Safe next action
`base/dot.json` or `prometheus/dot.json`	Validate and inspect the existing file. Do not rerun its creation block over your edits. Complete missing edits in your editor, then validate again.
`prometheus.yml`, Grafana inputs or a candidate	Open and review the existing file. Creation blocks refuse replacement; they do not reset an exercise for you. An incomplete, empty directory may be removed with `rmdir` before retrying; never delete a directory containing your work.
`composition.yaml` or `dashboard.yaml`, with its reviewed lock	Use the original file and `dotx compose status` with the restored repository. After an interrupted apply, use that lesson's destroy command before retrying. Do not create another UUID or regenerate a lock to adopt leftover resources.
A successful destroy and verified cleanup	Runtime state is gone; `COMPOSITION_STATE_MISSING` is expected. That missing record alone does not prove resource absence. Keep the learning files rather than rerunning creation blocks; the composition completion gate still applies before later live lessons.

If cleanup fails or Docker cannot be queried, cleanup is **unverified**. Keep the original files, cache and binary, restore Docker access, and retry the same cleanup command. Do not prune unrelated resources.

If a command failed, use the troubleshooting table and instance-scoped log instructions below. Never change a digest just to get past a validation error.

Troubleshoot this lab

The log block below is for this lesson's standalone Prometheus instance, using its original `DOT_NAME`. For a composition, retain that graph's draft, lock and cache and follow its lesson's recovery instructions instead.

Symptom	Action
Release or clone returns 404/403	Check `gh auth status` and organization repository access, including SSO authorization if required. Run `gh auth setup-git` for HTTPS cloning.
CLI missing or wrong version	Check `command -v dotx`; put `$HOME/.local/bin` first on `PATH`.
Docker permission denied	Check `docker info` as your user. Ask the administrator for the intended Docker access, then log in again. Docker group access grants host-level power.
Compose unavailable	Install the Compose plugin/package for your Docker distribution and verify `docker compose version`. The legacy `docker-compose` executable is insufficient.
Port already allocated	Clean up the failed example, select a free `DOTX_HTTP_PORT`, and deploy again with the same name and explicit `--env` override.

Symptom	Action
Contract download or digest check fails	Check network/proxy access to the declared URL. Never bypass its digest. Private raw GitHub URLs require `MARE_READ_ONLY_TOKEN` or `GITHUB_TOKEN`; `gh` login alone does not supply those to `dotx`. Use `--offline` only with matching documents already in the verified local cache.
Readiness or version fails	Use the instance-scoped logs below and `dotx ps --all`, check the selected image/port, then clean up. No local Compose file is required for this diagnostic.
Interrupted cleanup	Restore the original name and catalogue path, repeat cleanup, and inspect resources carrying `com.docker.compose.project=$DOT_NAME`.

Read only this instance's logs

Keep the original `DOT_NAME`. This block discovers containers by their Compose project label and reads bounded logs. It works from the catalogue checkout: `docker compose -p ⋯` alone would not locate the descriptor's downloaded Compose document. No resources are changed or removed.

```
(
set -euo pipefail
containers="$(docker ps -aq \
  --filter "label=com.docker.compose.project=$DOT_NAME")"
if test -z "$containers"; then
  printf 'No containers found for this instance: %s\n' "$DOT_NAME" >&2
  exit 1
fi
while IFS= read -r container; do
  printf '\nLogs for this instance container: %s\n' "$container"
  docker logs --tail=100 "$container"
done <<< "$containers"
)
```

If Docker access fails or no matching container is found, the block fails; it does not query another instance. Use the same lesson's cleanup command and retain the original context until cleanup is verified.

Checkpoint

Continue when you have:

- Seen the official API report Prometheus `3.15.1`.
- Seen the explicit cleanup-verification message.
- Saved your private workspace path and kept the original CLI and cache.

▼ *Explain it back: what did the dry-run prove?*

It checked and previewed the selected deployment without starting the service. The later API call established the running service's response and version. Successful cleanup established that this standalone run's resources were gone. These are three separate observations.

Previous: DOTs for Dummies · **All lessons** · **Next: create a real DOT and understand its model.**

2. Create an interface-aware DOT

Now build the definition yourself. You will keep the real product identity `Prometheus@3.15.1`, use its official OpenAPI document and bind two named Integration Points to a real service port. This is a **private learning copy**, not a second catalogue entry or a renamed fork of Prometheus.

Continue in the same Bash shell from Lesson 1, with `LAB` and `DOT_REPOSITORY` set. No runtime is started in this lesson.

Goal: create `$LAB/prometheus/dot.json` with verified interfaces and two deployment choices. **You will learn:** Contract ownership, provided/required Anchors and service-scoped configuration. Keep the generated scaffold so you can review exactly what the enrichment changes.

Author the provided interfaces

Reuse the catalogue's reviewed image digest and official Contract URL/hash. The CLI creates identifiers and the complete generated-model document:

```
(
set -euo pipefail
IMAGE="$(jq -r '.runtimes[].deployments[] |
  select(.id == "docker") | .manifest.image' "$PROMETHEUS_JSON")"
CONTRACT_URL="$(jq -r '.contracts[0].payload.url' "$PROMETHEUS_JSON")"
CONTRACT_SHA256="$(jq -r '.contracts[0].payload.sha256' \
  "$PROMETHEUS_JSON")"

printf '%s\n' \
  'Creating a private Prometheus definition' \
  'from the pinned official OpenAPI document...'
dotx new --from docker-image "$IMAGE" \
  --name Prometheus --version 3.15.1 --skill Metrics-Scraping \
  --service prometheus --port http:9090 \
  --contract-url "$CONTRACT_URL" --contract-sha256 "$CONTRACT_SHA256" \
  --contract-name HTTP --contract-standard OpenAPI \
  --contract-format YAML \
  --operation 'build-information=#/paths/~1status~1buildinfo/get' \
  --operation 'promql-query-api=#/paths/~1query/get' --bind http \
  --output "$LAB/base/dot.json"

dotx validate "$LAB/base/dot.json" \
  --repository "$DOT_REPOSITORY" --profile reusable
```

```
dotx inspect "$LAB/base/dot.json"
printf 'Draft verified: two provided Anchors.\n'
printf 'Follow their Contract references in %s/base/dot.json\n' "$LAB"
)
```

For Java integrations, see the current digest-reference decoding limit. Keep the image pin; this exercise uses the CLI.

Expected: an owned interface is verified, the output is saved, and reusable validation succeeds. `Draft verified: two provided Anchors` is your checkpoint. Open `$LAB/base/dot.json` in your editor. You supplied the release and selected operations; the CLI constructed their readable IDs and service bindings.

What you see	What it means here
`id`, `name`, `version`	One real tool release: `Prometheus@3.15.1`. This is not the CLI version.
`skills`	`Metrics-Scraping` is a capability reference, not an API operation.
`contracts`	A complete owned declaration. `payload.url` points to raw official bytes; `sha256` pins them. Contract version `3.1.0` is the OpenAPI standard version.
`anchors`	`build-information` and `promql-query-api` provide selected operations from that Contract. Their selectors are JSON Pointers, separate from Contract IDs.
`runtimes[].platform`	The `DOCKER` engine: where this deployment runs.
`deployments[].id` and `manifest`	`docker-image` selects a `DOCKER_IMAGE` mechanism and exact image bytes.
`services`, `ports`, `anchorBindings`	Service `prometheus` materializes both Anchors on the named `http` port, 9090. The flag `http:9090` supplies a name and number. Native authoring binds TCP listeners; the name `http` alone does not change that protocol label.
`metadata`	Definition audit timestamps. Live endpoints and lifecycle state are elsewhere.

For example, one generated Anchor looks like this:

```
{
  "id": "build-information",
  "name": "build-information",
  "role": "PROVIDES",
  "contract": {
    "id": "Prometheus@3.15.1::OpenAPI::HTTP",
    "selector": "#/paths/~1status~1buildinfo/get"
  }
}
```

`~1` encodes `/` inside a JSON Pointer segment. It selects the operation from verified Contract bytes; a documentation hyperlink would not establish an API match. The image pin and document pin establish identity and integrity, not publisher authenticity by themselves: the catalogue's provenance still matters.

Describe what Prometheus needs

An API provider can also be a consumer. Prometheus needs an exporter endpoint and a scrape configuration file. Its catalogue definition already documents that requirement and the real configuration path. Reuse those release-matched parts instead of inventing another interface. Also add its existing Compose deployment to see how one DOT supports more than one mechanism:

```
(
set -euo pipefail
set -o noclobber
mkdir "$LAB/prometheus"
jq --slurpfile product "$PROMETHEUS_JSON" '
  .anchors += [$product[0].anchors[] | select(.id == "scrape-target")]
  | .runtimes[0].deployments[0].services[0].ports[0].protocol = "HTTP"
  | .runtimes[0].deployments[0].services[0].config =
      ($product[0].runtimes[].deployments[] | select(.id == "docker")
       | .services[] | select(.id == "prometheus") | .config)
  | .runtimes[0].deployments +=
      [$product[0].runtimes[].deployments[]
       | select(.id == "docker-compose")]
' "$LAB/base/dot.json" > "$LAB/prometheus/dot.json"

dotx validate "$LAB/prometheus/dot.json" \
  --repository "$DOT_REPOSITORY" --profile reusable
dotx inspect "$LAB/prometheus/dot.json"
printf 'Definition verified: %s\n' \
  'three Anchors and two Docker deployment choices.'
printf 'Review the changes below.\n'
diff -u "$LAB/base/dot.json" "$LAB/prometheus/dot.json" ||
  test "$?" -eq 1
)
```

Read the diff. There are four intentional changes:

1. `scrape-target` is a **REQUIRES** Anchor referencing Node Exporter's exact Contract and originating DOT. Prometheus does not copy or own that Contract.
2. The named service port is labelled `HTTP`, the actual application protocol. This matters when deriving an HTTP URL for another tool; the network transport still uses TCP. The initial native scaffold uses a TCP label.
3. The image deployment declares `/etc/prometheus/prometheus.yml` as a required service configuration file. The composition will supply its contents.
4. The same Docker Runtime gains the catalogue's `docker-compose` deployment, including its

verified URL payload and declared defaults.

The final `$LAB/prometheus/dot.json` has three Anchors and two deployments. It is complete for the interfaces taught here; it is not a copy of every capability Prometheus could have. `DOCKER_IMAGE` and `DOCKER_COMPOSE` are mechanisms on the same engine. A Kubernetes deployment would belong in another Runtime.

The Prometheus service now declares the required file `/etc/prometheus/prometheus.yml`; Lesson 3 supplies its contents for your monitoring use case. The general rules for defaults and service-scoped configuration are in DOTs for Dummies.

Try the guided editor

This optional exercise changes declarations only. From the same shell:

```
dotx edit "$LAB/prometheus/dot.json" --interactive \
  --output "$LAB/prometheus.candidate.json" \
  --repository "$DOT_REPOSITORY"
```

Choose **Skills**, then **Add catalogue capability**. Filter for `Observability`, select it, then choose **Review and save**. Review the summary and explicitly choose **Write draft**. Arrows browse, typing filters, Enter selects, Ctrl-U clears a text field and Ctrl-C cancels. `ACCESSIBLE=1` selects line menus. The default publication choice is Cancel. In that fallback, `/text` filters, the displayed number selects, and `q` cancels.

Compare the candidate to the original. Unchanged Anchor IDs, manifests and configuration stay intact. The candidate is not automatically promoted; the remaining lessons deliberately use `$LAB/prometheus/dot.json`. Try another edit with an unused output path and cancel: no second candidate should appear.

The editor handles interfaces and Skills; it is not a manifest/configuration editor. The explicit JSON enrichment above is therefore a reviewed model edit, followed by validation, not an implied editor feature. For a genuinely new tool, use its own release, image and interfaces rather than relabelling these ones.

▼ *Inspect more detail when you need it*

The included commands use concise inspection to keep the next action visible. For the full declared configuration and bindings, run:

```
dotx inspect "$LAB/prometheus/dot.json" --verbose
```

This is still declaration inspection; no runtime is started.

If a creation block says the destination exists, your earlier work is protected. Inspect that existing file and follow the resume/retry guidance; do not remove an edited definition just to make the block run again.

Checkpoint

Continue when reusable validation passes for `$LAB/prometheus/dot.json` and inspection shows three Anchors and both `docker-image` and `docker-compose`. The optional editor candidate is separate; the next lesson uses the enriched `prometheus/dot.json` you just validated.

▼ *Explain it back: where does the future scrape target belong?*

The DOT declares the required interface and the supported configuration path. The composition selects a concrete exporter and supplies the scrape file. Its Wire materializes that exporter's address into the execution copy of the file. Changing the monitoring use case does not require another product identity.

Previous lesson · **All lessons** · **Next: make this authored DOT consume real metrics.**

3. Connect two DOTs

Your Prometheus definition has both provided and required interfaces. Connect its `scrape-target` to the catalogue's Node Exporter, then query the result. Continue in the same shell; Lesson 1's standalone service must already be stopped.

Goal: make `up{job="node-exporter",tutorial="zero-to-hero"}` return `1`. **You will learn:** exact instance selection, a provided-to-required Wire, connection configuration and a reviewed lock. Your final graph has **two DOTs and one Wire**.

Create the use-case configuration

This file belongs to **your monitoring use case**, not to the shared product. The label will let you distinguish your configured scrape from an arbitrary successful Prometheus response:

```
(
set -euo pipefail
set -o noclobber
cat > "$LAB/prometheus.yml" <<'YAML'
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: node-exporter
    static_configs:
      - targets: [node-exporter:9100]
        labels:
          tutorial: zero-to-hero
YAML
chmod 0644 "$LAB/prometheus.yml"
)
```

The file is non-secret. Mode 0644 lets the unprivileged Prometheus container read the mounted file; its enclosing workspace stays private. Do not apply these permissions to secret files. The initial

target is a placeholder: the Wire will write the selected instance's real host and port into the execution copy.

Create a fresh graph from the reviewed example

```
MONITORING_TEMPLATE="$DOT_REPOSITORY/Compositions"
MONITORING_TEMPLATE+="/node-exporter-prometheus-monitoring"
MONITORING_TEMPLATE+="/composition.yaml"
dotx compose new --from "$MONITORING_TEMPLATE" \
  --repository "$DOT_REPOSITORY" --name zero-to-hero \
  --output "$LAB/composition.yaml" --yes
```

Expected: a new UUID and a two-DOT, one-Wire preview. The template and its lock stay untouched. No lock is created for your draft and no service is deployed. The template's configuration references are initially shared; redirect them before editing a configuration.

Read the draft before changing it:

Field	Purpose in this graph
`apiVersion`, `kind`	Identify the generated Composition format, not a tool release or runtime.
`id`, `name`, `version`	A fresh ownership UUID, a readable use-case name and this graph's version. The graph's `1.0.0` is separate from each tool's release.
`dots`	Named instances, exact tool identities, selected deployments and instance inputs. `prometheus` is an instance name; `Prometheus@3.15.1` is its tool identity.
`contracts`	Verified interface inputs resolved from their exact owning DOTs, not copied specifications.
`wires`	Connections and the configuration needed to materialize them.
`exposedAnchors`	Names you can select with `--forward`, such as `prometheus-query`. An exposure does not add authentication.

Open `$LAB/composition.yaml` in a text editor and make **two changes**:

1. Replace only the `prometheus` entry under `dots` with:

   Keep the four leading spaces shown before `- name`, matching the other DOT item in the CLI-created draft. YAML indentation is part of the document.

```
    - name: prometheus
      source: ./prometheus
      dotId: Prometheus@3.15.1
      dotVersion: 3.15.1
      deployment: docker-image
      dependsOn: [node-exporter]
```

Before, this entry selected `repository://Prometheus@3.15.1` and deployment `docker`. After, `./prometheus` and `docker-image` select your private authored definition. Its identity, tool release and readiness dependency stay the same.

2. In `node-metrics-to-prometheus`, change `materializer.source` to `prometheus.yml`. Keep the other Wire fields unchanged.

   Before, the source points to the template's `runtime/prometheus.yml` (the draft may show its full absolute path). After, it names your own file beside the draft:

   ```
   materializer:
     type: CONFIG_FILE
     source: prometheus.yml
   ```

   This last snippet shows the changed field's location, not the complete materializer: retain its existing `valueFrom` and `target` below these lines.

`./prometheus` selects **your authored definition**, not the catalogue's deployment. The local `docker-image` ID comes from your `dotx new` output. Leave Node Exporter and the composition's Contract inputs at their exact catalogue coordinates. The owned Prometheus Contract has the same verified bytes as that release's catalogue declaration. Relative file paths resolve beside this YAML file.

Understand the connection

The retained Wire names these model relationships:

```
from: {instance: node-exporter, anchorId: node-metrics}
to: {instance: prometheus, anchorId: scrape-target}
materializer:
  type: CONFIG_FILE
  source: prometheus.yml
  valueFrom: ENDPOINT_HOST_PORT
  target:
    instance: prometheus
    service: prometheus
    path: /etc/prometheus/prometheus.yml
    jsonPointer: /scrape_configs/0/static_configs/0/targets/0
```

`from` provides metrics; `to` requires them. The materializer derives the producer's address and places

it at one location in the consumer's configuration. It does not implement the scrape: Prometheus does that. `dependsOn` expresses startup ordering; a Wire expresses an interface connection.

In the pointer `/scrape_configs/0/static_configs/0/targets/0`, each `0` means the first item in a list. Follow `scrape_configs` to its first job, then `static_configs` to its first entry, then `targets` to its first value: `node-exporter:9100` in your file. The materializer replaces that scalar with the selected producer's address in the execution copy; your source file is kept as the input.

Node Exporter's metrics operation uses a **source-derived OpenAPI projection**. It is not a vendor-published OpenAPI specification or complete OpenMetrics certification. Both sides select that same exact interface. The unprivileged exporter observes its container; this exercise makes no physical-host claim.

Validate, lock and preview

These are deliberate review steps: validate the edited declaration, create its lock, inspect the exact resolved inputs and topology, then preview deployment. The block stops if any step fails; it starts no services.

```
(
set -euo pipefail
dotx compose validate "$LAB/composition.yaml" \
  --repository "$DOT_REPOSITORY"
dotx compose lock "$LAB/composition.yaml" --repository "$DOT_REPOSITORY"
dotx compose inspect "$LAB/composition.yaml" \
  --repository "$DOT_REPOSITORY" --locked
dotx compose topology "$LAB/composition.yaml" \
  --repository "$DOT_REPOSITORY" --locked
dotx compose apply "$LAB/composition.yaml" \
  --repository "$DOT_REPOSITORY" \
  --locked --dry-run --forward "prometheus-query=$TUTORIAL_PORT"
printf 'Graph verified: two DOTs, one Wire. No service has started.\n'
)
```

Expected: two DOTs, one Wire and a deployment preview using your local `docker-image` definition. Review `$LAB/composition.lock.json`: it binds exact definitions, documents and configuration. A lock is reproducibility evidence, not runtime state or a second authored model. Intentional input changes require another explicit lock review; never regenerate locks just to hide an error.

Open the lock and check the selections

Where to look	Expected fields and meaning
`dots[]`, entry with `instance: "prometheus"`	`source: "./prometheus"`, `dotId: "Prometheus@3.15.1"`, `deploymentId: "docker-image"`, and a `digest` beginning `sha256:` Your private definition and selected deployment are pinned by their bytes.
`dots[]`, entry with `instance: "node-exporter"`	`source: "repository://Node-Exporter@1.12.1"`, `deploymentId: "docker"`, and a `digest` The exact catalogue release is selected.

Where to look	Expected fields and meaning
`inputs[]`, entry with `source: "prometheus.yml"`	A `name` beginning `wire:node-metrics-to-prometheus:config:` and a `digest` The Wire's materializer uses the pinned input scrape configuration. This does not establish that a scrape has run.

Array order and digest values can differ. Relative source paths may be normalized (`prometheus` and `./prometheus` select the same folder here). Stop if the selected sources or deployments differ: review your edits before regenerating the lock. Do not edit the lock by hand. These entries describe resolved inputs; the next section checks actual running services.

Run and ask a meaningful question



On the pinned CLI `0.4.0` route, stop here before live apply. Read this section as a preview until the composition completion gate is satisfied by a released, qualified CLI. Do not switch an existing lab's binary.

Use an unused `TUTORIAL_PORT` (default `19041`). Before applying, keep the stop command below available. **Run it even if apply or a query fails.** Preserve the original workspace, cache and binary for recovery.

```
printf '%s\n' \
  'Starting Node Exporter and your Prometheus;' \
  'the next check waits for their labelled scrape...'
dotx compose apply "$LAB/composition.yaml" \
  --repository "$DOT_REPOSITORY" \
  --locked --forward "prometheus-query=$TUTORIAL_PORT"
```

```
(
set -euo pipefail
set -o pipefail
scraped=false
started=$SECONDS
next_notice=$SECONDS
printf '%s\n' \
  'Checking the labelled Node Exporter scrape through Prometheus...'
for ((attempt=0; attempt<90; attempt++)); do
  if curl --fail --silent --max-time 3 --get \
    --data-urlencode \
    'query=up{job="node-exporter",tutorial="zero-to-hero"}' \
    "http://127.0.0.1:$TUTORIAL_PORT/api/v1/query" |
    jq -e '.status == "success" and
      any(.data.result[]?; .value[1] == "1")' >/dev/null; then
    scraped=true
    break
  fi
  if ((SECONDS >= next_notice)); then
    printf 'Waiting for the scrape: attempt %s/90, %ss elapsed\n' \
```

```
      "$((attempt + 1))" "$((SECONDS - started))"
    next_notice=$((SECONDS + 5))
  fi
  sleep 1
done
if ! $scraped; then
  printf '%s\n' \
    'Scrape check timed out. Inspect compose status, then run' \
    'the Lesson 3 cleanup block before retrying.' >&2
  exit 1
fi
curl --fail --silent --show-error --max-time 5 \
  "http://127.0.0.1:$TUTORIAL_PORT/api/v1/status/buildinfo" |
  jq -e '.status == "success" and .data.version == "3.15.1"'
dotx compose status "$LAB/composition.yaml" \
  --repository "$DOT_REPOSITORY"
printf 'Verified: your labelled Node Exporter scrape is up (value 1).\n'
printf 'Try the same query in Prometheus: http://127.0.0.1:%s/\n' \
  "$TUTORIAL_PORT"
)
```

Expected: the check reports progress while waiting, build information returns `true`, status lists the selected instances, and a `Verified:` message confirms the labelled scrape. The loop permits up to 90 attempts with three-second HTTP timeouts and one-second pauses. Open the URL printed by the check (Prometheus with the default port) and evaluate:

```
up{job="node-exporter",tutorial="zero-to-hero"}
```

The result `1` establishes your labelled scrape, not merely an HTTP 200. The label came from your configuration; the target address came from the Wire. The original catalogue and its configuration are unchanged.

Stop before extending

```
dotx compose destroy "$LAB/composition.yaml" \
  --repository "$DOT_REPOSITORY"
```

`Composition destroyed:` reports removal. It is not the resource-absence check required by this book. On the pinned `0.4.0` route, that completion gate remains unavailable; do not mark cleanup verified or continue to a fresh live graph from that message alone. If you already started this graph, run the original destroy command and retain its draft, lock, cache and binary while cleanup is unresolved. The lab's time-series data is disposable. Do not prune unrelated Docker resources or delete by guessed names. A later `COMPOSITION_STATE_MISSING` only means the lifecycle record is gone.

Try a safe diagnostic exercise: copy the YAML to a new file, change `from.anchorId` to `missing-metrics`, and validate that copy **without** `--locked`. Expect a missing-Anchor rejection. Restore the

reference and validate again. Do not modify the reviewed live draft or publish another lock for the bad copy.

If the result differs

Symptom	What to check before retrying
`INPUT_YAML`	Check the edited list item's four leading spaces and the surrounding list. Do not add a second `dots:` key. Validate again before creating a lock.
A missing Anchor or a lock mismatch	Compare the exact instance, Anchor and local definition references with the reviewed edit. If an input changed intentionally, inspect it before explicitly recreating the lock. A new lock cannot repair an invalid reference.
No sample, or a sample with value `0`	`0` means the target's scrape failed; no sample means the selected series is absent. Check lifecycle status, the `node-exporter` job, the tutorial label and the Wire's materializer source. The API build-information check alone is insufficient.
A readiness or query timeout	Stop and use this lesson's destroy command. Keep the original files, cache and binary if cleanup fails; use the recovery guide.

Checkpoint

Continue after the labelled query returns `1`, the topology shows two DOTs and one Wire, and cleanup has established absence of the owned resources. The pinned `0.4.0` route cannot satisfy the last condition using the native audit. Keep the draft, lock and scrape file; do not start a fresh live graph while cleanup is unverified. Lesson 4's draft uses a fresh ownership UUID.

▼ *Explain it back: which direction does this connection run?*

The Wire points from Node Exporter's provided Anchor to Prometheus's required Anchor. At runtime, Prometheus sends an HTTP scrape request to Node Exporter. The Wire describes the interface relationship; the application implements the request. The materializer supplies its address through configuration.

Previous lesson · **All lessons** · **Next: extend this same system with Grafana.**

4. Grow the system with Grafana

You now have a useful two-tool system. Add a dashboard without rewriting either tool. This lesson extends your existing graph: **Node Exporter → your Prometheus → Grafana**. It adds one DOT instance, one Wire and a browser-facing exposure.

Continue in the same Bash shell. First complete Lesson 3's destroy step. The new graph gets another ownership UUID; it does not adopt the stopped graph's resources or inherit its live state.

Goal: see the same labelled scrape through Grafana. **You will learn:** extending a graph,

provisioning a datasource and supplying a password through a reference. The result is **three DOTs and two Wires**; the original scrape relationship stays intact.

Prepare the next real tool

Reuse Grafana's exact catalogue release in a private exercise copy. Add a service-scoped declaration for its supported admin-password environment option:

```
(
set -euo pipefail
set -o noclobber
GRAFANA_JSON="$(dotx repository resolve Grafana@13.3.0 \
  --repository "$DOT_REPOSITORY")"
mkdir "$LAB/grafana"
jq '.runtimes[0].deployments[0].services[0].config.environment +=
  [{"name":"GF_SECURITY_ADMIN_PASSWORD",
    "required":true,"sensitive":true}]' \
  "$GRAFANA_JSON" > "$LAB/grafana/dot.json"
dotx validate "$LAB/grafana/dot.json" \
  --repository "$DOT_REPOSITORY" --profile reusable

cat > "$LAB/grafana-prometheus.yml" <<'YAML'
apiVersion: 1
datasources:
  - name: Prometheus
    uid: hero-prometheus
    type: prometheus
    access: proxy
    url: http://replaced-by-the-wire.invalid
    isDefault: true
YAML
cat > "$LAB/grafana-empty.yml" <<'YAML'
apiVersion: 1
datasources: []
YAML
chmod 0644 "$LAB/grafana-prometheus.yml" "$LAB/grafana-empty.yml"

dotx compose new --from "$LAB/composition.yaml" \
  --repository "$DOT_REPOSITORY" --name hero-dashboard \
  --output "$LAB/dashboard.yaml" --yes
)
```

Expected: the Grafana candidate validates and `dashboard.yaml` retains the earlier monitoring graph with a fresh UUID. The catalogue remains unchanged. The copied product already carries its official pinned OpenAPI declaration, required Prometheus Anchor, provided dashboard Anchor and service bindings. You extended its **configuration surface**, not its API identity.

The datasource file is an instance input. `uid: hero-prometheus` makes it easy to select and test. `access: proxy` makes Grafana's backend query Prometheus over the composition network; the

browser does not need to reach an internal container address. The Wire replaces the `.invalid` URL before deployment.

This catalogue release also requires an Elasticsearch provisioning file. The empty, valid `grafana-empty.yml` deliberately provisions no Elasticsearch datasource: that integration is outside this lab. Do not draw a fictitious Wire or claim that it has been tested.

Extend the existing graph

Open `$LAB/dashboard.yaml`. Before editing, it has two DOTs and one Wire. You will add one item to each of three **existing** lists. Keep the earlier items, Contract inputs and ownership UUID; do not paste another root `dots:` or `wires:` key. Each snippet below is a list item, beginning with `- name`, at the same indentation as the existing items. Keep the four leading spaces shown before each `- name` in the CLI-created draft.

Add Grafana under `dots`

Append this after the existing `prometheus` instance:

```
  - name: grafana
    source: ./grafana
    dotId: Grafana@13.3.0
    dotVersion: 13.3.0
    deployment: docker
    dependsOn: [prometheus]
    deploymentInputs:
      environment:
        - name: GF_SECURITY_ADMIN_PASSWORD
          secretRef: {store: environment, name: HERO_GRAFANA_PASSWORD}
      configFiles:
        - source: grafana-empty.yml
          target: /etc/grafana/provisioning/datasources/elasticsearch.yml
```

Add the second connection under `wires`

Append this after `node-metrics-to-prometheus`:

```
    - name: prometheus-to-grafana
      from: {instance: prometheus, anchorId: promql-query-api}
      to: {instance: grafana, anchorId: promql-query-api}
      materializer:
        type: CONFIG_FILE
        source: grafana-prometheus.yml
        valueFrom: ENDPOINT_URL
        target:
          instance: grafana
          service: grafana
          path: /etc/grafana/provisioning/datasources/prometheus.yml
          jsonPointer: /datasources/0/url
```

Add the browser endpoint under `exposedAnchors`

Append this after `prometheus-query`:

```
    - name: dashboard
      target: {instance: grafana, anchorId: monitoring-dashboard}
```

The new Wire connects a provided PromQL operation to Grafana's required operation. Its materializer writes the selected Prometheus base URL into `/datasources/0/url`. The two Wires have different jobs: the first configures scraping; the second configures querying.

Here `/datasources/0/url` selects `url` in the first datasource entry. It replaces the `.invalid` placeholder URL you created in `grafana-prometheus.yml` with this graph's Prometheus base URL.

`secretRef` names a host environment variable; it contains no password. The executor resolves it at apply time. Do not paste a sensitive `value` into the composition. Docker environment delivery is still visible to an authorized Docker administrator; it is not a vault or native Kubernetes Secret.

```
(
set -euo pipefail
dotx compose validate "$LAB/dashboard.yaml" \
  --repository "$DOT_REPOSITORY"
dotx compose lock "$LAB/dashboard.yaml" --repository "$DOT_REPOSITORY"
dotx compose topology "$LAB/dashboard.yaml" \
  --repository "$DOT_REPOSITORY" --locked
printf 'Extended graph verified: three DOTs, two Wires.\n'
printf 'Next: supply a lab password, then preview and apply.\n'
)
```

Expected: **three DOTs, two Wires**. Open `$LAB/dashboard.lock.json` and use Lesson 3's lock-field checklist. Keep the local Prometheus and catalogue Node Exporter selections. The additional `dots[]` entry must have `instance: "grafana"`, `source: "./grafana"`, `dotId: "Grafana@13.3.0"`, `deploymentId: "docker"` and a definition `digest`. The `inputs[]` now also pin `grafana-prometheus.yml` and `grafana-empty.yml`. Creating or inspecting the lock does not require the password; never paste a secret value

into it.

Set a password and start the extended system



On the pinned CLI `0.4.0` route, stop before the live apply block below. The composition completion gate remains open. Read the launch and checks as a preview; do not switch binaries in an existing lab.

Use a new lab-only password. The prompt does not echo it or put it in shell history. Keep this environment and the same binary for the lifecycle:

```
if read -r -s -p 'Choose a lab-only Grafana admin password: ' \
  HERO_GRAFANA_PASSWORD &&
  test -n "$HERO_GRAFANA_PASSWORD"; then
  printf '\n'
  export HERO_GRAFANA_PASSWORD
else
  unset HERO_GRAFANA_PASSWORD
  printf '\nNo password set. Repeat this prompt before applying.\n' >&2
  false
fi
```

Ports `19041` and `13000` must be free. Use only the isolated development host and network described at the start. Keep the cleanup command below available before applying; use it after a failure as well. The next block previews both exposures first; a failed preview prevents its apply command from running. Runtime preflight, including the dry-run, needs the referenced password available.

```
printf '%s\n' \
  'Previewing, then starting Node Exporter, Prometheus and Grafana;' \
  'pinned images may need downloading...'
dotx compose apply "$LAB/dashboard.yaml" \
  --repository "$DOT_REPOSITORY" --locked --dry-run \
  --forward "prometheus-query=$TUTORIAL_PORT" \
  --forward "dashboard=$DASHBOARD_PORT" &&
dotx compose apply "$LAB/dashboard.yaml" \
  --repository "$DOT_REPOSITORY" --locked \
  --forward "prometheus-query=$TUTORIAL_PORT" \
  --forward "dashboard=$DASHBOARD_PORT"
```

Verify the whole chain

Grafana health is the first check. The datasource query is the second: it exercises the connection to Prometheus and the labelled scrape from Node Exporter. Complete both before calling the chain successful.

Wait for Grafana's actual health/version response:

```
(
set -euo pipefail
set -o pipefail
dashboard_ready=false
started=$SECONDS
next_notice=$SECONDS
printf 'Checking Grafana 13.3.0 health on port %s...\n' \
  "$DASHBOARD_PORT"
for ((attempt=0; attempt<90; attempt++)); do
  if curl --fail --silent --max-time 3 \
    "http://127.0.0.1:$DASHBOARD_PORT/api/health" |
    jq -e '.database == "ok" and .version == "13.3.0"' >/dev/null; then
    dashboard_ready=true
    break
  fi
  if ((SECONDS >= next_notice)); then
    printf 'Waiting for Grafana: attempt %s/90, %ss elapsed\n' \
      "$((attempt + 1))" "$((SECONDS - started))"
    next_notice=$((SECONDS + 5))
  fi
  sleep 1
done
if ! $dashboard_ready; then
  printf '%s\n' \
    'Grafana health check timed out. Inspect compose status, then run' \
    'the Lesson 4 cleanup block before retrying.' >&2
  exit 1
fi
printf 'Ready: Grafana 13.3.0 at http://127.0.0.1:%s/\n' \
  "$DASHBOARD_PORT"
)
```

Expected: named waiting messages include elapsed time, followed by `Ready: Grafana 13.3.0` and the browser URL. Its loop permits up to 90 attempts, with three-second HTTP timeouts and one-second pauses.

Open Grafana, sign in as `admin` with your chosen password, and open **Explore**. Choose the provisioned **Prometheus** datasource and run:

```
up{job="node-exporter",tutorial="zero-to-hero"}
```

Expect a sample with value 1. Grafana UI labels can vary by release; the backend check below provides a reproducible alternative. You have extended one running use case without putting a use-case-specific datasource or password into the shared product definitions.

▼ *Verify the same query without a browser*

Make Grafana's backend query the provisioned datasource. The temporary curl credentials file is private and removed when the check finishes, including a failed query. Neither the password nor

its value enters YAML or the lock.

```
(
set -euo pipefail
set -o pipefail
# curl reads credentials from a private file, not command-line arguments.
credentials="$(mktemp "$LAB/grafana.curl.XXXXXXXX")"
trap 'rm -f -- "$credentials"' EXIT
(umask 077; jq -nr '
  "user = " + ("admin:" + env.HERO_GRAFANA_PASSWORD | tojson)
' > "$credentials")
dashboard_queried=false
started=$SECONDS
next_notice=$SECONDS
printf 'Querying the labelled scrape through Grafana...\n'
for ((attempt=0; attempt<90; attempt++)); do
  if curl --config "$credentials" --fail --silent --max-time 5 \
    --header 'Content-Type: application/json' \
    --data '{
      "from":"now-5m", "to":"now",
      "queries":[{
        "refId":"A",
        "expr":"up{job=\"node-exporter\",tutorial=\"zero-to-hero\"}",
        "instant":true,
        "datasource":{"type":"prometheus","uid":"hero-prometheus"}
      }]
    }' \
    "http://127.0.0.1:$DASHBOARD_PORT/api/ds/query" |
    jq -e '.results.A.status == 200 and
      any(.results.A.frames[]?.data.values[]?[]?; . == 1)' \
      >/dev/null; then
    dashboard_queried=true
    break
  fi
  if ((SECONDS >= next_notice)); then
    printf 'Waiting for the Grafana query: attempt %s/90, %ss\n' \
      "$((attempt + 1))" "$((SECONDS - started))"
    next_notice=$((SECONDS + 5))
  fi
  sleep 1
done
if ! $dashboard_queried; then
  printf '%s\n' \
    'Grafana query timed out. Check the password and datasource;' \
    'run the Lesson 4 cleanup block before retrying.' >&2
  exit 1
fi
printf 'Verified: Grafana queried the labelled scrape (value 1).\n'
)
```

Expected: `Verified: Grafana queried the labelled scrape (value 1).` The bounded loop allows 90 attempts with five-second HTTP timeouts and one-second pauses. The native request selects datasource UID `hero-prometheus` and runs an instant query. This verifies Grafana → Prometheus → Node Exporter; the diagram alone could not establish it.

Clean up the complete graph

```
dotx compose destroy "$LAB/dashboard.yaml" \
  --repository "$DOT_REPOSITORY" &&
unset HERO_GRAFANA_PASSWORD &&
{
  if test -f "$LAB/grafana.curl"; then
    rm -- "$LAB/grafana.curl"
  fi
} &&
printf 'Destroy succeeded: keep the workspace files for reference.\n'
```

Expected: all three owned instances stop. This is disposable storage; downloaded images remain cached. Keep your authored files as a learning reference, but retain the original cache and lock if cleanup did not finish. Do not remove unrelated resources or change a lock to adopt an existing instance. The final message and password removal happen only after destroy succeeds. `Composition destroyed:` and the final message do not independently prove resource absence, and neither does removing the password from this shell. The pinned `0.4.0` lacks the required native audit: keep the original binary, draft, lock and cache while cleanup is unverified. A missing lifecycle record alone does not close that gate. If it fails, retain the current environment and use the recovery steps before retrying.

Checkpoint

Finish when Grafana reports version `13.3.0`, its datasource query returns the labelled sample with value `1`, and cleanup has established absence of the complete graph's owned resources. The pinned `0.4.0` route cannot satisfy that last condition using the native audit; do not mark the full lab complete. Either Explore or the documented backend query can verify that sample; the health response alone cannot.

If sign-in or the backend query fails, check that you used the lab password prompt, datasource UID `hero-prometheus` and the chosen dashboard port. A resumed running instance needs its original password. Use the recovery guide before retrying an interrupted run.

▼ *Explain it back: what stays reusable when you add a dashboard?*

Prometheus keeps its product identity, Contract and Anchors. Grafana's datasource, the selected Prometheus instance, the password reference and both Wires belong to this composition. The private Grafana copy adds a supported sensitive configuration declaration; it contains no password value. Live addresses and resolved credentials remain execution concerns.

Previous lesson · **All lessons** · **Next: apply the pattern to larger, secured systems.**

5. Take the pattern further

The preceding lessons describe a real chain: **Node Exporter → your Prometheus → Grafana**. They take you through authoring, reuse, connected interfaces, configuration, a secret reference and live checks. On the pinned `0.4.0` route, the composition completion gate limits Lessons 3 and 4 to previews before live apply. Do not claim their live results or verified cleanup from reading them.

If you have run the Docker compositions, confirm removal of their owned resources before leaving the lab; a successful destroy message alone is not independent absence proof. Repeat the original cleanup command only for an interrupted or unresolved run; missing lifecycle state is expected after successful cleanup. Keep the private learning files if useful. Do not publish the exercise copies as additional Prometheus or Grafana catalogue roots.

What you can now do

After completing a qualified live route, you will have practiced this loop:

1. **Describe** a release with Skills, verified Contracts and named Anchors.
2. **Select** a deployment and declare its service configuration.
3. **Connect** provided and required Anchors through reviewed Wires.
4. **Review** exact inputs with validation, a lock and a deployment preview.
5. **Verify** an actual API result, then clean up the owned resources.

Keep your final topology and any observed successful query results as a learning record. Only actual live checks establish those particular interactions. Production work also needs the security, persistence, operations and interface coverage in the production integration checklist.

Grow deliberately

Your next goal	What changes	What must be verified
Observe more targets	Select more instances and configure additional targets; keep reusable product definitions shared. Give each materialization its own reviewed configuration location.	Each intended target is actually scraped; labels distinguish producers; interruption and recovery affect only the selected instance.
Use another runtime	Choose a matching deployment from the same tool release. Review its service configuration, native workload selection and required permissions.	Readiness, real API behavior and cleanup on that adapter. The Docker/Kubernetes example is deployment portability, not proof of a cross-runtime Wire.

Your next goal	What changes	What must be verified
Cross a runtime boundary	Supply an explicitly provisioned external-gateway host/port using the existing execution profile. The operator retains gateway ownership.	Reachability from the actual consumer and attribution to the exact producer. There is no automatic infrastructure or protocol translation.
Package a multi-service tool	Declare each logical service's ports, config and Anchor bindings. Bind services to exact native workloads at composition level when needed.	Separate configuration and secrets, same-Pod/container selection, peer isolation and ownership-safe cleanup. See services and inputs.
Use messaging or another API style	Select verified AsyncAPI, GraphQL, Protobuf or SPARQL operations through guided authoring. Broker transport endpoints are separate inputs.	The supported operation/transport profile, authentication and actual exchange. Four-standard test fixtures are not production-producer certification.

Operator capstone: authentication and policy

The next difficulty is controlling who can use an API. The existing Keycloak/APISIX/OPA/SPARQL composition provides a five-DOT exercise:

- Keycloak provides identity and token operations.
- APISIX routes a protected capability.
- OPA decides whether a request is allowed.
- Oxigraph provides the SPARQL query service.
- A clearly labelled demo client exercises the authenticated path.

This is a **separate advanced graph**, not a claim that the preceding Grafana lab has already been secured or connected to it. Keep your tested monitoring system as the small reference while learning native Secret references, policy, multiple Wires and service startup. Only combine graphs after defining and testing the actual additional interfaces and ownership boundaries.

This capstone requires a qualified operator environment, Kubernetes, native Secrets and the matching source toolchain/artifact set. It is not a hidden prerequisite for the Docker tutorial. From the **DOT-Design checkout root**, after following the contributor setup and the secure-API prerequisites, the existing presenter route is:

```
task testing:pocs:present:secure-api
```

Expected: anonymous rejection, authenticated success, the expected RDF result and policy checks, followed by an inspection pause and owned cleanup. The full qualification route also exercises failure and interruption cleanup. Local HTTP defaults, synthetic clients and explicit-interface boundaries remain documented; this is not complete MARE producer acceptance.

Contributor note: native cleanup audit

The current-source CLI includes a native Docker cleanup audit in `dotx compose destroy`. Its `Cleanup verified:` message follows successful absence checks for recorded containers, managed networks and disposable volumes. Verification failure retains recovery state. This behavior is not provided by released `0.4.0`; the practical book's release pin must be advanced and its exact commands rehearsed before this becomes the supported newcomer route. A local source rehearsal does not close that release dependency.

Turn a learning result into a contribution

1. For an existing product release, propose changes to its one canonical definition. For another supported release, use its exact release folder and release-matched artifacts and Contracts; see catalogue layout.
2. Keep use-case files in a Composition. Document its purpose, prerequisites, selected deployments, topology, expected API results, cleanup and limitations.
3. Validate against DOT-Domain's generated schemas, inspect and review changed locks, then rehearse the affected real interactions and failure cleanup.
4. Keep credentials and runtime state out of Git. Treat official and source-derived specifications honestly; a hash does not establish origin.

For exact CLI options use `dotx <command> --help`. For deeper explanations use DOT authoring, composition authoring, the complete model reference and production integration limits.

Contributors can rehearse the same included command blocks from DOT-Design with `task testing:tutorial` (declarations/previews) and `task testing:tutorial:live` (actual Docker services and cleanup). The test driver applies the documented editor changes, rather than maintaining a second tutorial definition. These are maintainer rehearsals, not independent newcomer-trial results.

Previous lesson · **All lessons**